

Surface Metrics Comparison: Human-AI vs Human-Human vs AI-AI

Cross-Condition Text Descriptives Analysis

PENPAL Project

2026-04-07

Contents

1 Overview	1
2 Load Data	1
3 Preprocessing	3
4 Descriptive Statistics	3
5 Cross-Condition Comparisons	5
5.1 Word Count	5
5.2 Lexical Diversity	6
5.3 Readability	7
6 Author-Level Comparison	8
7 Visualization Summary	9
8 Key Findings	10

1 Overview

This notebook compares surface-level text features across experimental conditions:

- **Human-AI:** Human co-writing with an LLM
- **Human-Human:** Two humans co-writing together
- **AI-AI:** Two AI agents co-writing (simulated baseline)

Key Metrics: - Word count per turn - Sentence length - Lexical diversity (type-token ratio) - Readability scores - Punctuation patterns

Key Questions: 1. Do AI contributions differ in length/complexity from human contributions? 2. Are Human-Human stories more varied than Human-AI or AI-AI? 3. Does the AI-AI baseline show characteristic “AI-like” text patterns?

2 Load Data

```
# Check data availability
cat("Data availability:\n")
```

```
## Data availability:
avail <- check_data_availability()
print(avail)

## # A tibble: 6 x 4
##   file                                `human-ai` `human-human` `ai-ai`
##   <chr>                                <lgl>      <lgl>      <lgl>
## 1 dyadic_sentiment_scores.parquet TRUE       TRUE       TRUE
## 2 sentiment_scores.parquet FALSE      FALSE      FALSE
## 3 surface_metrics.parquet TRUE       FALSE      FALSE
## 4 semantic_exploration.parquet FALSE      FALSE      FALSE
## 5 embeddings.parquet FALSE      FALSE      FALSE
## 6 cleaned_stories.csv FALSE      FALSE      FALSE

# Load surface metrics
df_combined <- tryCatch({
  load_surface_metrics(conditions = c("human-ai", "human-human", "ai-ai"))
}, error = function(e) {
  cat("Error loading surface metrics:", e$message, "\n")
  cat("Attempting to load from individual conditions...\n")

  conditions_available <- c()
  for (cond in c("human-ai", "human-human", "ai-ai")) {
    path <- here("data", cond, "processed", "interaction_level_surface_metrics.parquet")
    if (file.exists(path)) {
      conditions_available <- c(conditions_available, cond)
    }
  }

  if (length(conditions_available) == 0) {
    stop("No surface metrics data available")
  }

  load_surface_metrics(conditions = conditions_available)
})

# Prepare condition factor
df_combined <- df_combined %>%
  prepare_condition_factor(reference = "human-ai")

# Report loaded data
cat("\nLoaded data summary:\n")

##
## Loaded data summary:
df_combined %>%
  group_by(condition) %>%
  summarise(
    n_stories = n_distinct(conversation_id),
    n_turns = n(),
    .groups = "drop"
  ) %>%
```

```
print()
```

```
## # A tibble: 3 x 3
##   condition  n_stories n_turns
##   <fct>      <int>   <int>
## 1 Human-AI      88    1760
## 2 Human-Human   38     760
## 3 AI-AI        40     800
```

3 Preprocessing

```
# Identify available metrics
# Common surface metrics from textdescriptives
potential_metrics <- c(
  # Length metrics
  "n_tokens", "n_words", "n_chars", "n_sentences",
  # Complexity
  "avg_word_length", "avg_sentence_length", "n_syllables",
  # Diversity
  "ttr", "mtld", "hdd",
  # Readability
  "flesch_reading_ease", "flesch_kincaid_grade", "gunning_fog",
  "smog", "automated_readability_index", "coleman_liau_index",
  # Other
  "n_unique_tokens", "entropy"
)

# Find which metrics are present
available_metrics <- intersect(potential_metrics, names(df_combined))
cat("Available metrics:", paste(available_metrics, collapse = ", "), "\n")
```

```
## Available metrics: n_tokens, n_sentences, flesch_reading_ease, flesch_kincaid_grade, gunning_fog, sm
```

```
# Ensure we have at least some metrics
if (length(available_metrics) == 0) {
  stop("No recognized surface metrics found in the data")
}
```

4 Descriptive Statistics

```
# Summary statistics by condition for key metrics
if ("n_tokens" %in% available_metrics || "n_words" %in% available_metrics) {
  word_col <- if ("n_words" %in% available_metrics) "n_words" else "n_tokens"

  desc_length <- df_combined %>%
    group_by(condition) %>%
    summarise(
      n_turns = n(),
      mean_words = mean(!is.na(word_col), na.rm = TRUE),
      sd_words = sd(!is.na(word_col), na.rm = TRUE),
      median_words = median(!is.na(word_col), na.rm = TRUE),
      .groups = "drop"
```

```

)

desc_length %>%
  knitr::kable(
    digits = 2,
    caption = "Word count by condition"
  )
}

```

Table 1: Word count by condition

condition	n_turns	mean_words	sd_words	median_words
Human-AI	1760	26.96	7.94	27
Human-Human	760	26.13	5.29	26
AI-AI	800	28.24	2.35	29

```

# Visualize distributions for key metrics
plot_metric_dist <- function(df, metric, title) {
  if (!metric %in% names(df)) return(NULL)

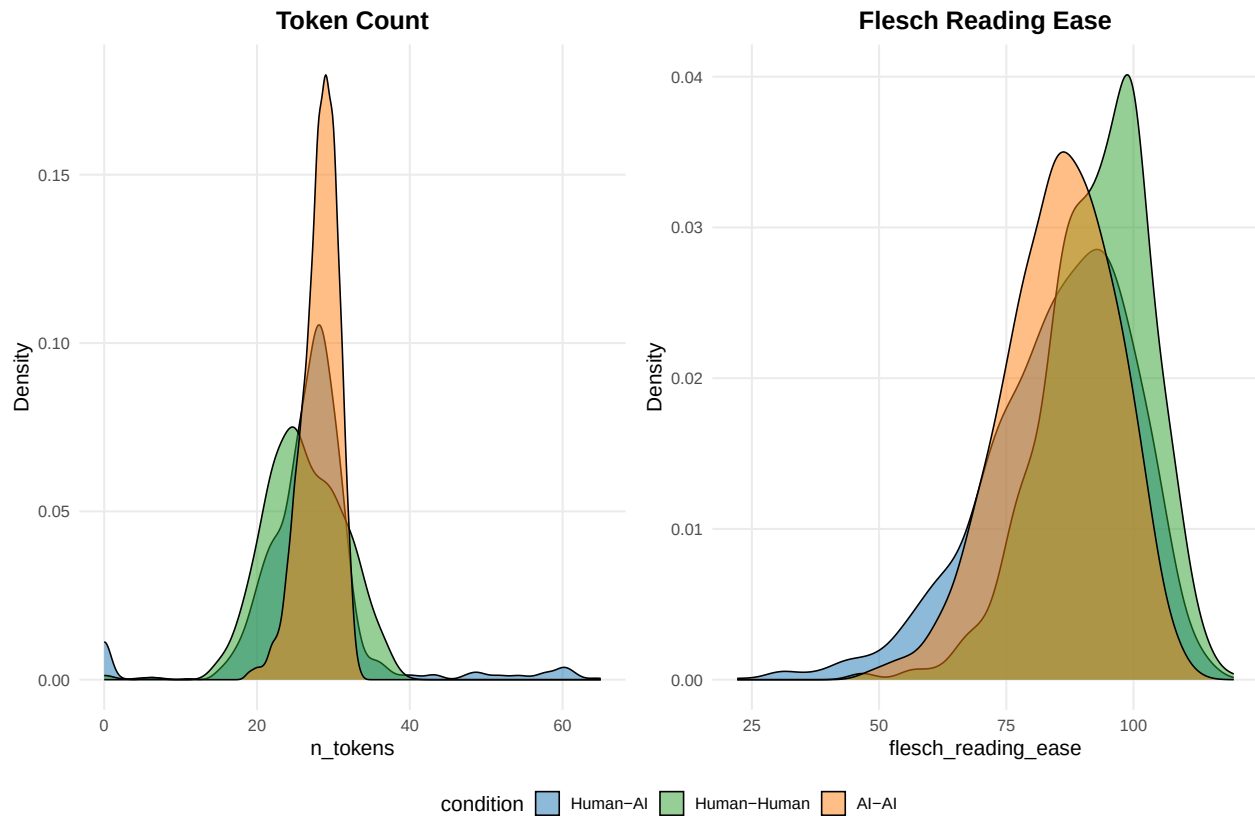
  ggplot(df, aes(x = !!sym(metric), fill = condition)) +
    geom_density(alpha = 0.5) +
    scale_fill_manual(values = condition_colors) +
    labs(title = title, x = metric, y = "Density") +
    theme(legend.position = "none")
}

# Create distribution plots for available metrics
plots <- list()

if ("n_words" %in% available_metrics) {
  plots$words <- plot_metric_dist(df_combined, "n_words", "Word Count")
}
if ("n_tokens" %in% available_metrics && !("n_words" %in% available_metrics)) {
  plots$tokens <- plot_metric_dist(df_combined, "n_tokens", "Token Count")
}
if ("avg_sentence_length" %in% available_metrics) {
  plots$sent_len <- plot_metric_dist(df_combined, "avg_sentence_length", "Avg Sentence Length")
}
if ("ttr" %in% available_metrics) {
  plots$ttr <- plot_metric_dist(df_combined, "ttr", "Type-Token Ratio")
}
if ("flesch_reading_ease" %in% available_metrics) {
  plots$flesch <- plot_metric_dist(df_combined, "flesch_reading_ease", "Flesch Reading Ease")
}

if (length(plots) > 0) {
  wrap_plots(plots, ncol = 2) +
    plot_layout(guides = "collect") &
    theme(legend.position = "bottom")
}

```



5 Cross-Condition Comparisons

5.1 Word Count

```
word_col <- if ("n_words" %in% available_metrics) "n_words" else
  if ("n_tokens" %in% available_metrics) "n_tokens" else NULL

if (!is.null(word_col)) {
  # Mixed model for word count
  model_words <- lmer(
    as.formula(paste(word_col, "~ condition + (1 | conversation_id)")),
    data = df_combined
  )

  cat("=== Word Count by Condition ===\n")
  print(summary(model_words))

  # Pairwise contrasts
  emm_words <- emmeans(model_words, specs = "condition")
  cat("\n=== Estimated Marginal Means ===\n")
  print(emm_words)

  cat("\n=== Pairwise Contrasts ===\n")
  print(pairs(emm_words, adjust = "bonferroni"))
}

## === Word Count by Condition ===
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: as.formula(paste(word_col, "~ condition + (1 | conversation_id)"))
## Data: df_combined
##
## REML criterion at convergence: 21127.2
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -8.1567 -0.3750  0.0794  0.4559  4.9446
##
## Random effects:
## Groups           Name          Variance Std.Dev.
## conversation_id (Intercept) 10.65     3.263
## Residual                30.67     5.538
## Number of obs: 3320, groups: conversation_id, 166
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)    26.9563    0.3720 163.0000  72.457  <2e-16 ***
## conditionHuman-Human -0.8299    0.6774 163.0000  -1.225   0.2223
## conditionAI-AI      1.2837    0.6655 163.0000   1.929   0.0555 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##              (Intr) cndH-H
## cndtnHmn-Hm -0.549
## condtnAI-AI -0.559  0.307
##
## === Estimated Marginal Means ===
## condition  emmean    SE df asymp.LCL asymp.UCL
## Human-AI    27.0 0.372 Inf     26.2     27.7
## Human-Human  26.1 0.566 Inf     25.0     27.2
## AI-AI        28.2 0.552 Inf     27.2     29.3
##
## Degrees-of-freedom method: asymptotic
## Confidence level used: 0.95
##
## === Pairwise Contrasts ===
## contrast              estimate    SE df z.ratio p.value
## (Human-AI) - (Human-Human)    0.83 0.677 Inf   1.225  0.6616
## (Human-AI) - (AI-AI)        -1.28 0.666 Inf  -1.929  0.1612
## (Human-Human) - (AI-AI)      -2.11 0.791 Inf  -2.674  0.0225
##
## Degrees-of-freedom method: asymptotic
## P value adjustment: bonferroni method for 3 tests
```

5.2 Lexical Diversity

```
if ("ttr" %in% available_metrics) {
  model_ttr <- lmer(
    ttr ~ condition + (1 | conversation_id),
```

```

    data = df_combined
  )

  cat("=== Type-Token Ratio by Condition ===\n")
  print(summary(model_ttr))

  emm_ttr <- emmeans(model_ttr, specs = "condition")
  cat("\n=== Pairwise Contrasts ===\n")
  print(pairs(emm_ttr, adjust = "bonferroni"))
}

```

5.3 Readability

```

if ("flesch_reading_ease" %in% available_metrics) {
  model_flesch <- lmer(
    flesch_reading_ease ~ condition + (1 | conversation_id),
    data = df_combined
  )

  cat("=== Flesch Reading Ease by Condition ===\n")
  print(summary(model_flesch))

  emm_flesch <- emmeans(model_flesch, specs = "condition")
  cat("\n=== Pairwise Contrasts ===\n")
  print(pairs(emm_flesch, adjust = "bonferroni"))
}

```

```

## === Flesch Reading Ease by Condition ===
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: flesch_reading_ease ~ condition + (1 | conversation_id)
## Data: df_combined
##
## REML criterion at convergence: 25579.2
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -4.8241 -0.6137  0.0800  0.6771  2.9068
##
## Random effects:
##  Groups             Name             Variance Std.Dev.
## conversation_id (Intercept)  33.85      5.818
## Residual                    133.10     11.537
## Number of obs: 3272, groups: conversation_id, 166
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)    85.3206    0.6799 163.5413 125.485 < 2e-16 ***
## conditionHuman-Human    7.5268    1.2365 162.7076   6.087 7.98e-09 ***
## conditionAI-AI         0.1361    1.2145 162.5595   0.112  0.911
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##

```

```
## Correlation of Fixed Effects:
##          (Intr) cndH-H
## cndtnHmn-Hm -0.550
## condtnAI-AI -0.560  0.308

##
## === Pairwise Contrasts ===
## contrast                estimate    SE  df z.ratio p.value
## (Human-AI) - (Human-Human)   -7.527  1.24 Inf  -6.087  <.0001
## (Human-AI) - (AI-AI)         -0.136  1.21 Inf  -0.112  1.0000
## (Human-Human) - (AI-AI)       7.391  1.44 Inf   5.125  <.0001
##
## Degrees-of-freedom method: asymptotic
## P value adjustment: bonferroni method for 3 tests
```

6 Author-Level Comparison

Compare Author 1 vs Author 2 contributions within each condition.

```
# Check if we have author-specific columns
author_1_cols <- names(df_combined)[str_detect(names(df_combined), "^author_1_")]
author_2_cols <- names(df_combined)[str_detect(names(df_combined), "^author_2_")]

if (length(author_1_cols) > 0 && length(author_2_cols) > 0) {
  cat("Author-level metrics available\n")

  # Find matching pairs
  author_1_metrics <- str_replace(author_1_cols, "^author_1_", "")
  author_2_metrics <- str_replace(author_2_cols, "^author_2_", "")
  common_metrics <- intersect(author_1_metrics, author_2_metrics)

  if ("n_words" %in% common_metrics || "n_tokens" %in% common_metrics) {
    word_metric <- if ("n_words" %in% common_metrics) "n_words" else "n_tokens"

    # Pivot to long format for comparison
    df_author <- df_combined %>%
      select(conversation_id, turn, condition,
             starts_with("author_1_"), starts_with("author_2_")) %>%
      pivot_longer(
        cols = c(paste0("author_1_", word_metric), paste0("author_2_", word_metric)),
        names_to = "author",
        values_to = "words"
      ) %>%
      mutate(author = if_else(str_detect(author, "author_1"), "Author 1", "Author 2"))

    # Plot
    ggplot(df_author, aes(x = author, y = words, fill = condition)) +
      geom_boxplot(alpha = 0.7, outlier.alpha = 0.1) +
      facet_wrap(~condition) +
      scale_fill_manual(values = condition_colors) +
      labs(
        title = "Word Count by Author and Condition",
        x = "Author",
        y = "Words per Turn"
      )
  }
}
```



```

    ) +
    theme(legend.position = "none")
  }
} else {
  cat("No author-level surface metrics found. Data may be at turn level without author separation.\n")
}

```

No author-level surface metrics found. Data may be at turn level without author separation.

7 Visualization Summary

```

# Compute summary statistics for available metrics
summary_stats <- df_combined %>%
  group_by(condition) %>%
  summarise(
    across(all_of(available_metrics[1:min(6, length(available_metrics))]),
      list(mean = ~mean(., na.rm = TRUE), se = ~sd(., na.rm = TRUE)/sqrt(n())),
      .names = "{.col}_{.fn}"),
    .groups = "drop"
  )

# Create bar plot for first few metrics
metrics_to_plot <- available_metrics[1:min(4, length(available_metrics))]

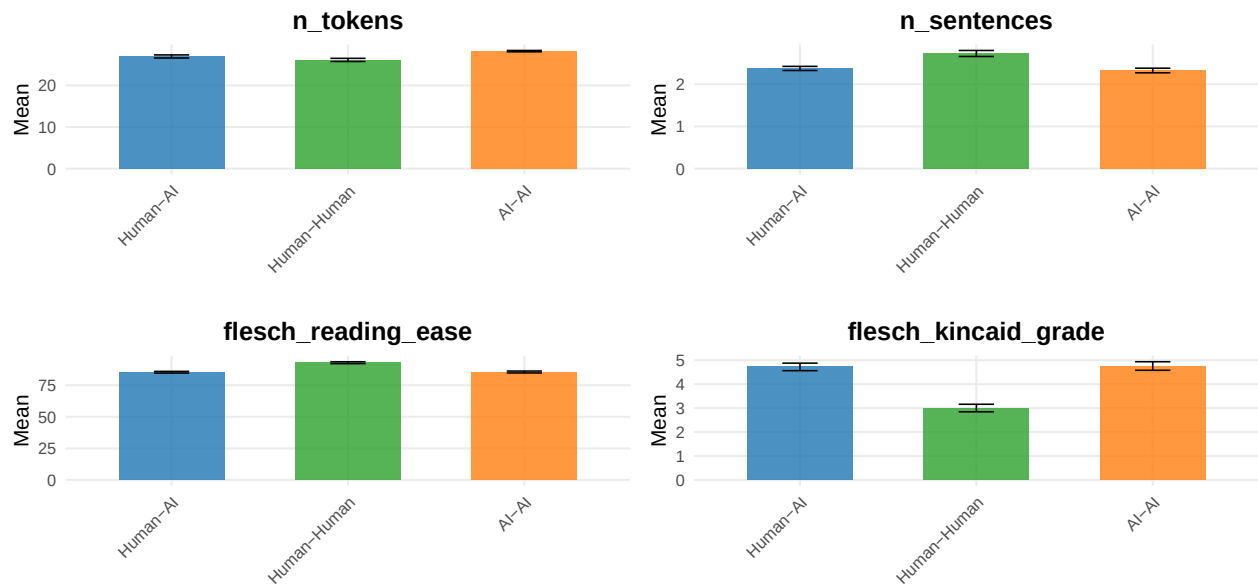
plots_summary <- lapply(metrics_to_plot, function(m) {
  mean_col <- paste0(m, "_mean")
  se_col <- paste0(m, "_se")

  if (!all(c(mean_col, se_col) %in% names(summary_stats))) return(NULL)

  ggplot(summary_stats, aes(x = condition, y = !!sym(mean_col), fill = condition)) +
    geom_col(alpha = 0.8, width = 0.6) +
    geom_errorbar(
      aes(ymin = !!sym(mean_col) - 1.96 * !!sym(se_col),
          ymax = !!sym(mean_col) + 1.96 * !!sym(se_col)),
      width = 0.2
    ) +
    scale_fill_manual(values = condition_colors) +
    labs(title = m, x = "", y = "Mean") +
    theme(legend.position = "none",
          axis.text.x = element_text(angle = 45, hjust = 1))
})

plots_summary <- compact(plots_summary)
if (length(plots_summary) > 0) {
  wrap_plots(plots_summary, ncol = 2)
}

```



8 Key Findings

Based on the analysis above:

1. **Word count patterns:** [Interpret whether AI contributions are longer/shorter than human]
2. **Lexical diversity:** [Compare vocabulary richness across conditions]
3. **Readability:** [Discuss complexity/accessibility differences]
4. **AI-AI characteristics:** [Note any distinctive patterns in simulated baseline]

Analysis completed: 2026-04-07 18:49:20.354212